Hundred Day's of Coding C programming

Experiment-4

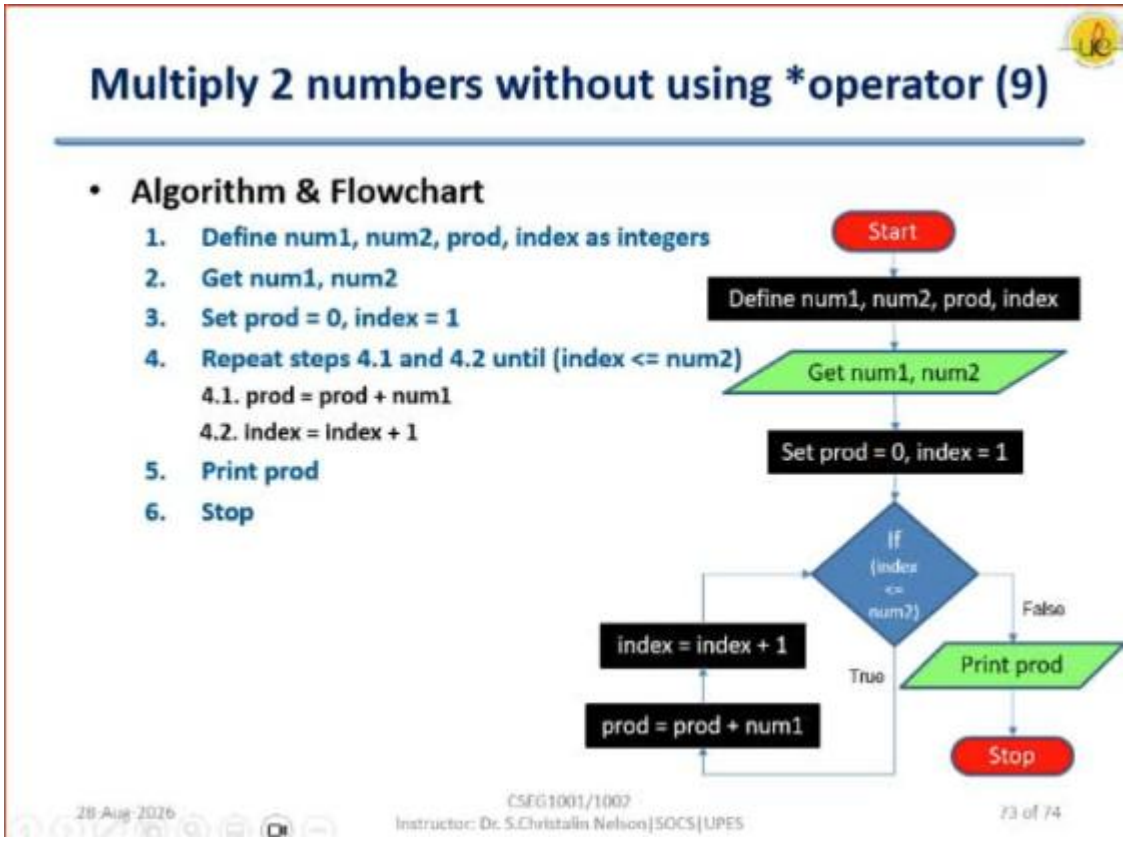
Name: Arayana Khare

Batch:15

Sap Id:590042432

Question

Find the incomplete algorithm & flowchart attaches below for today's HDOC Question.



Question 1

Identify the mistakes in the given algorithm.

Mistakes in the given algorithm

I found a few mistakes in the given algorithm:

1. **Mistake in Step 4:**
   It says **“Repeat until (index <= num2)”**. This is not correct because the loop should continue as long as index is less than or equal to num2.
   So, it should be **“Repeat while (index <= num2)”**.

2. **The flowchart is slightly confusing:**
   In the flowchart, after checking index <= num2, the steps are shown as index = index + 1 followed by prod = prod + num1. It would be clearer to first add num1 to prod and then increase the index.

3. **Negative numbers are not considered:**
   The algorithm works properly for positive numbers, but it will not give the correct answer if num2 is negative.

Question 2

4. **Corrected Step 4**

Repeat steps 4.1 and 4.2 while (index <= num2)

4.1 prod = prod + num1
4.2 index = index + 1

Question 3

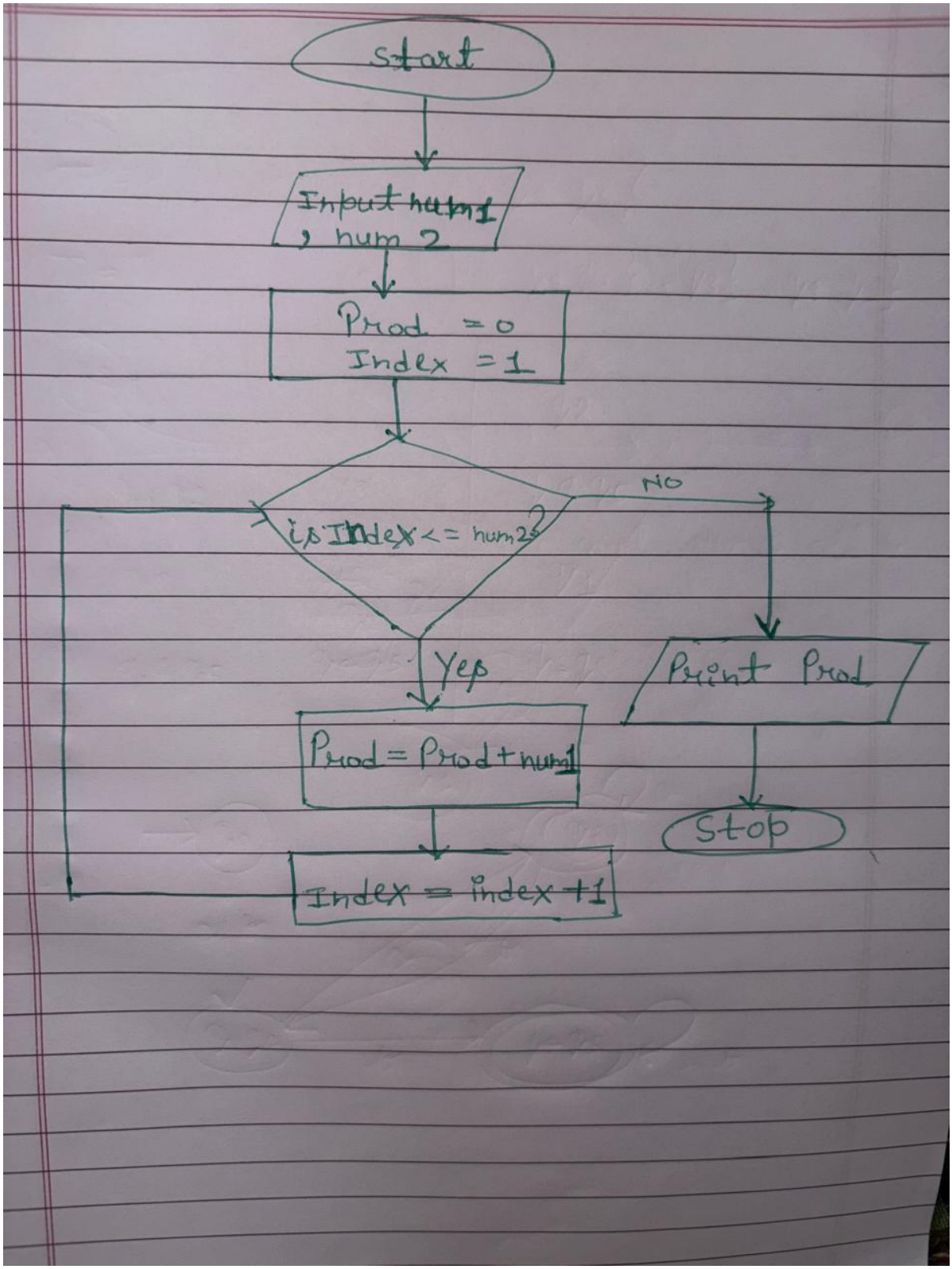
Find the mistakes in the flow chart.

1. The flowchart does not handle negative numbers properly.
2. It does not handle negative * negative correctly.
3. The condition index <= num2 does not work when num2 is negative.
4. There is no proper check for zero.
5. It does not always use the minimum number of additions.
6. It always uses num2 for repetition, instead of choosing the smaller number for fewer additions.

Question 4

Correct Flow Chart

Corrected flow chart.



Question

The following problem is to find the product of two integers without using the multiplication operator *.

Your teacher has given an incorrect algorithm and an incorrect flowchart for this problem (Refer Slideshare).

Study both carefully and complete the tasks given below. [Hint: Use repeated addition]

Requirements:

The logic must use the minimum possible number of additions and should correctly handle:

- positive × positive
- positive × negative
- negative × positive
- negative × negative
- multiplication by zero

Tasks

- Identify the mistakes in the given algorithm.
- Rewrite the corrected algorithm.
- Identify the mistakes in the given flowchart.
- Redraw the corrected flowchart.
- Verify your corrected algorithm and flow chart using the given test cases.

Test Case	Input	Expected Output	Minimum Additions
1	12 3	Product: 36	3
2	3 12	Product: 36	3
3	-6 5	Product: -30	5
4	6 -5	Product: -30	5
5	-9 -4	Product: 36	4
6	-4 -9	Product: 36	4
7	0 8	Product: 0	0
8	8 0	Product: 0	0
9	0 -7	Product: 0	0
10	-7 0	Product: 0	0
11	0 0	Product: 0	0
12	1 -7	Product: -7	1
13	-1 -9	Product: 9	1

main.c

```c
#include <stdio.h>

int main()
{
    int num1, num2;
    int a, b;
    int prod = 0;
    int sign = 1;
    int count;

    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    if (num1 == 0 || num2 == 0)
    {
        printf("Product: 0\n");
        return 0;
    }

    a = num1;
    b = num2;

    if (a < 0)
    {
        sign = -sign;
        a = -a;
    }

    if (b < 0)
    {
        sign = -sign;
        b = -b;
    }

    if (a > b)
    {
        int temp = a;
        a = b;
        b = temp;
    }

    count = 0;

    while (count < a)
    {
```

```
46          prod = prod + b;
47          count = count + 1;
48      }
49
50      if (sign < 0)
51          prod = -prod;
52
53      printf("Product: %d\n", prod);
54
55      return 0;
56  }
```

Test Case Explanation

Case 1: 12 3

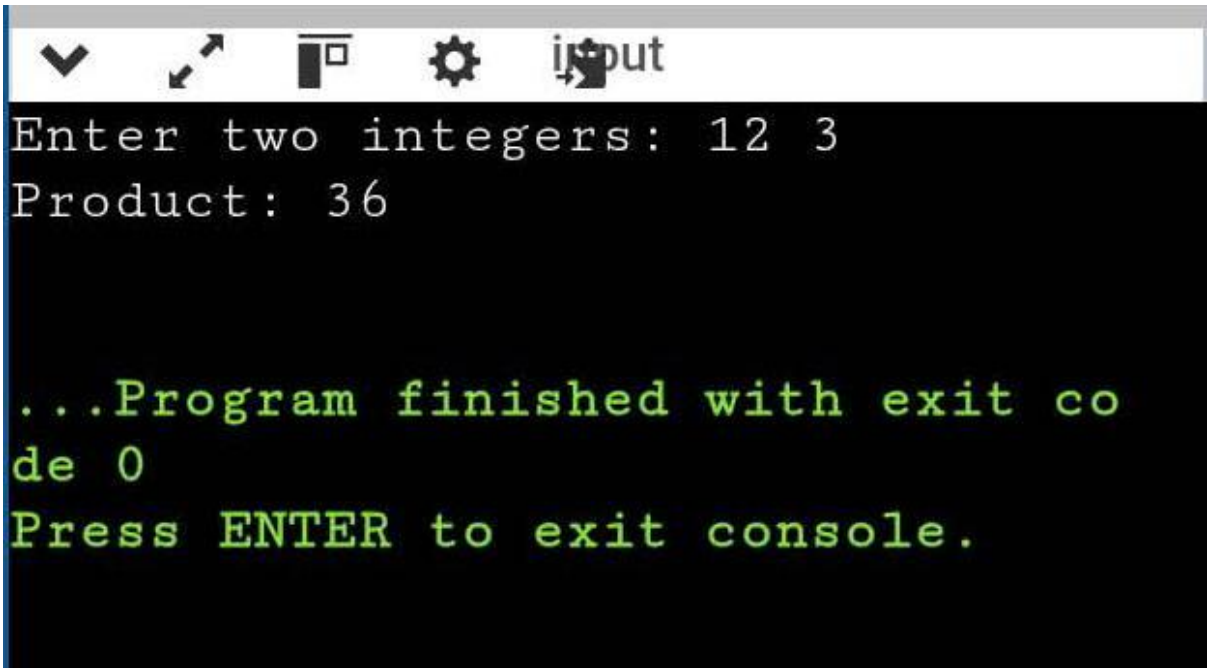
Both numbers are positive.

Smaller number is 3, so we add 12 three times.

12 + 12 + 12 = 36

Output: 36

Additions: 3



Case 2: 3 12

Both numbers are positive.

Smaller number is 3.

We add 12 three times.

12 + 12 + 12 = 36

Output: 36

Additions: 3

Case 3: -6 5

First number is negative.

We take its positive value as 6 and remember that the answer should be negative.

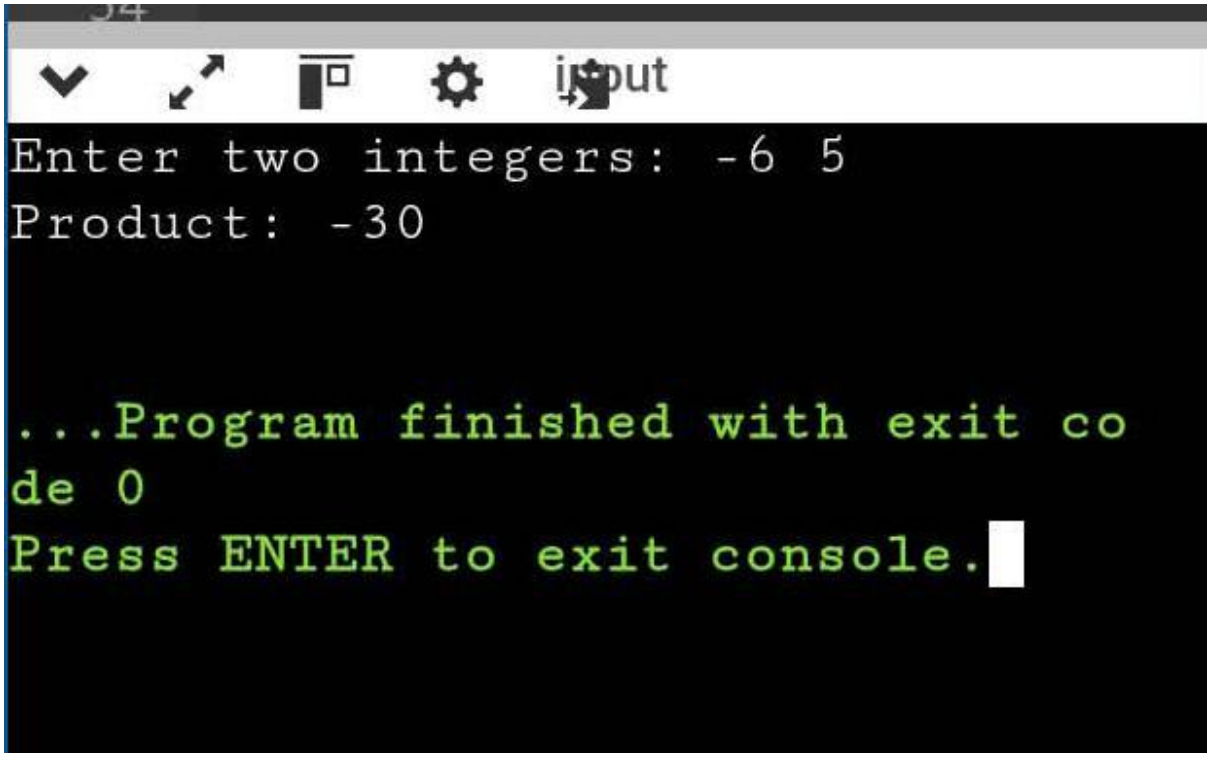
We add 6 five times.

6 + 6 + 6 + 6 + 6 = 30

So the answer is -30.

Output: -30

Additions: 5



Case 4: 6 -5

Second number is negative.

We take its positive value as 5 and keep the negative sign.

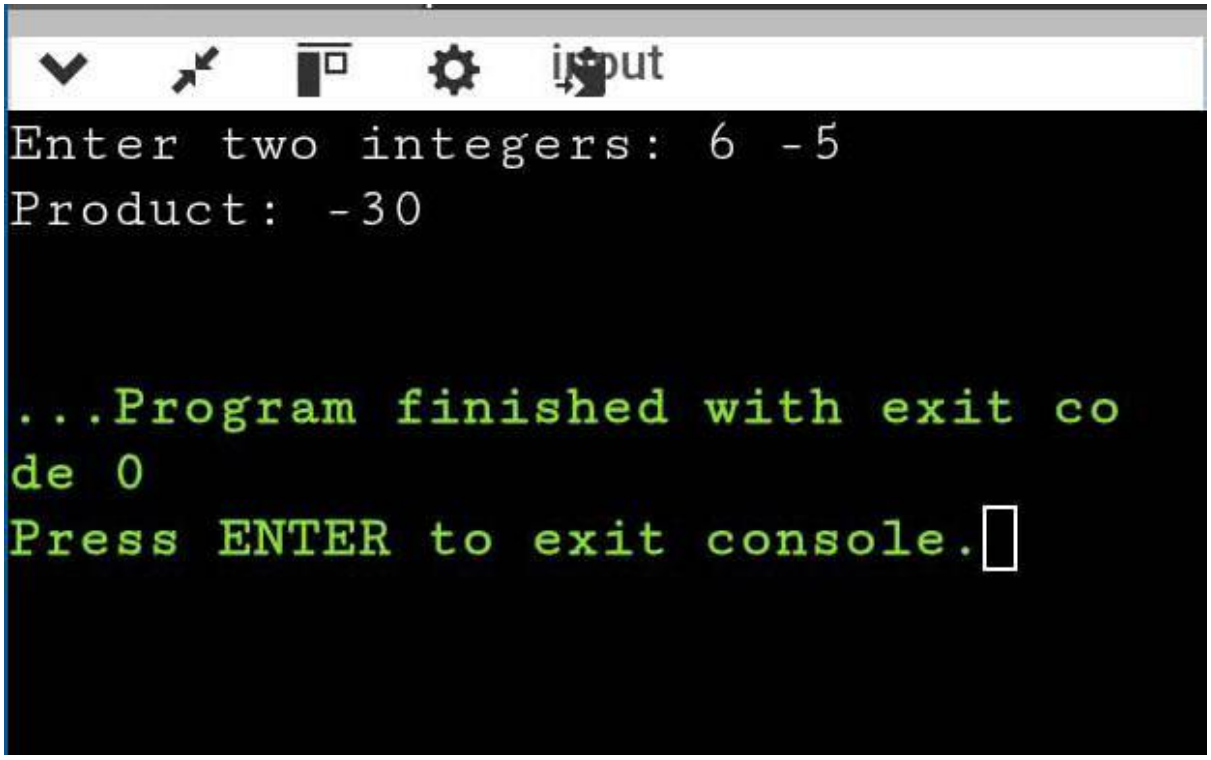
We add 6 five times.

6 + 6 + 6 + 6 + 6 = 30

Final answer is negative.

Output: -30

Additions: 5



Case 5: -9 -4

Both numbers are negative.

Negative signs cancel each other, so the answer will be positive.

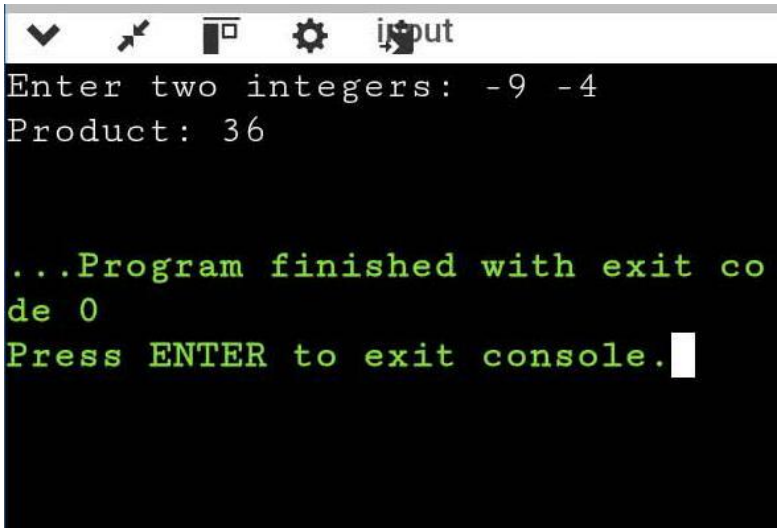
Smaller number is 4.

We add 9 four times.

9 + 9 + 9 + 9 = 36

Output: 36

Additions: 4



Case 6: -4 -9

Both numbers are negative, so the answer will be positive.

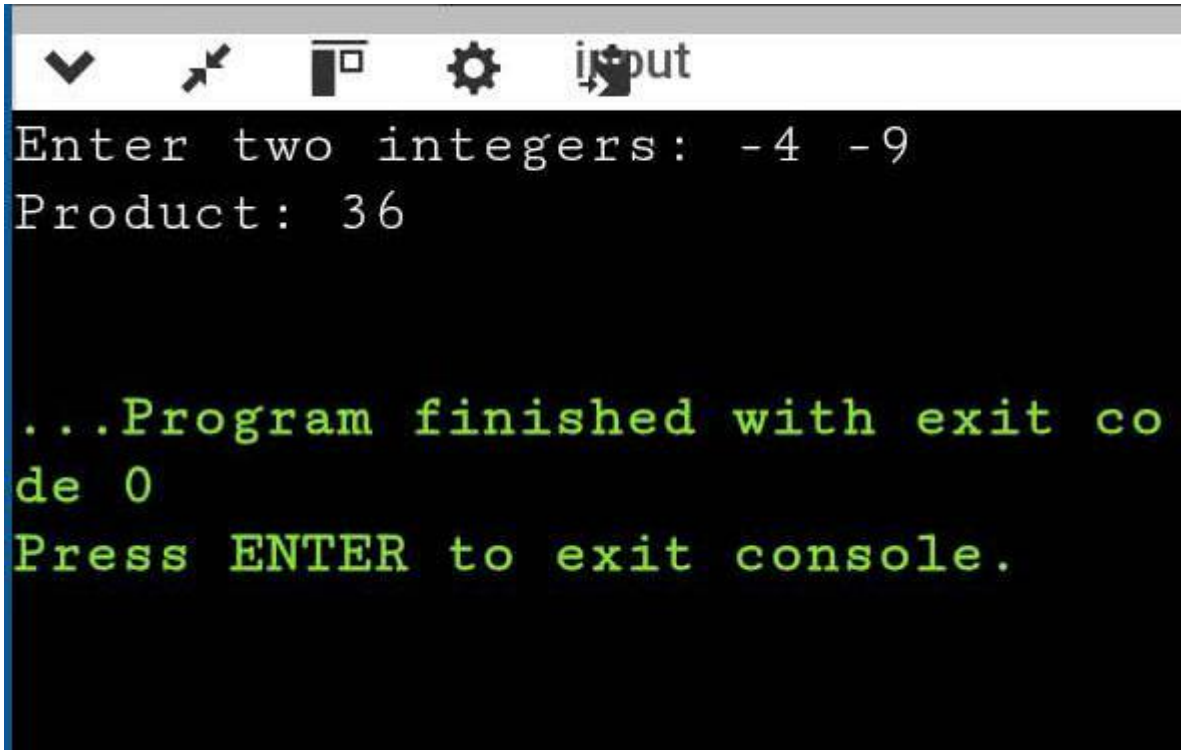
Smaller number is 4.

We add 9 four times.

9 + 9 + 9 + 9 = 36

Output: 36

Additions: 4



Case 7: 0 8

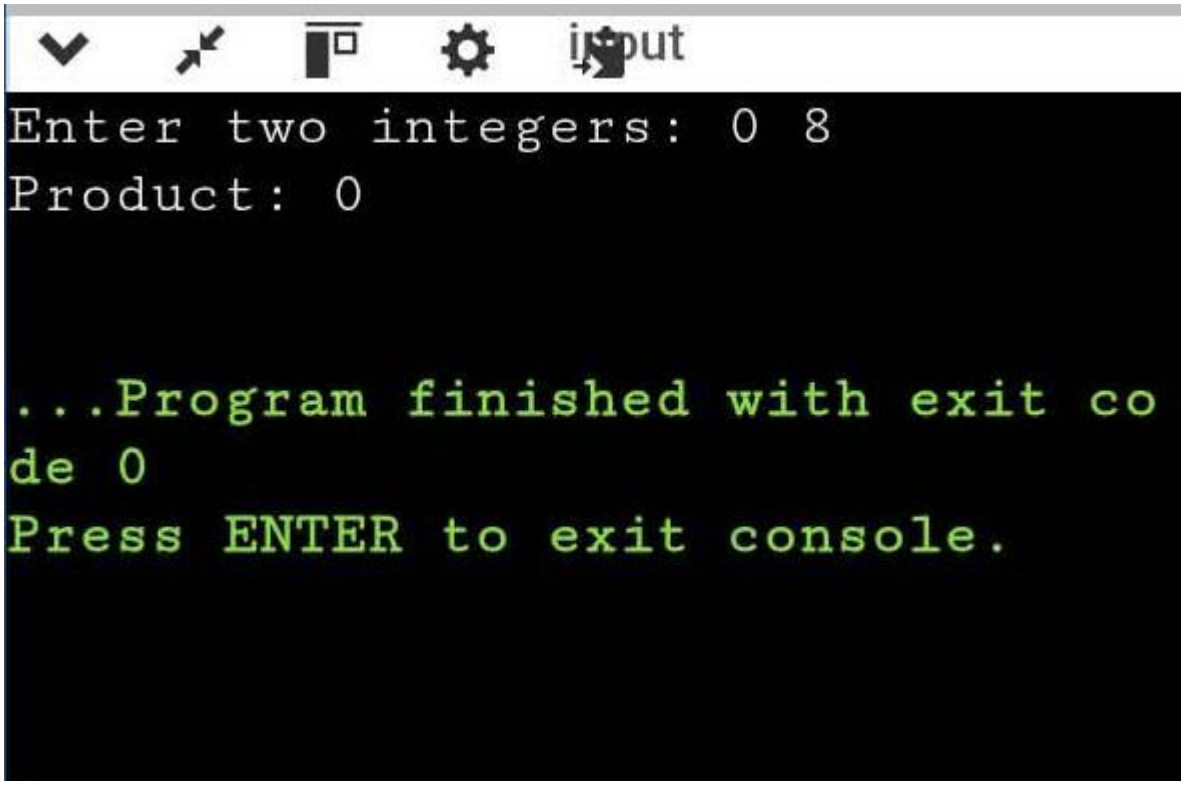
One number is zero.

Any number multiplied by zero gives zero.

So no addition is required.

Output: 0

Additions: 0



Case 8: 8 0

Second number is zero.

Therefore, the product is zero.

No loop or addition is required.

Output: 0

Additions: 0

Case 9: 0 -7

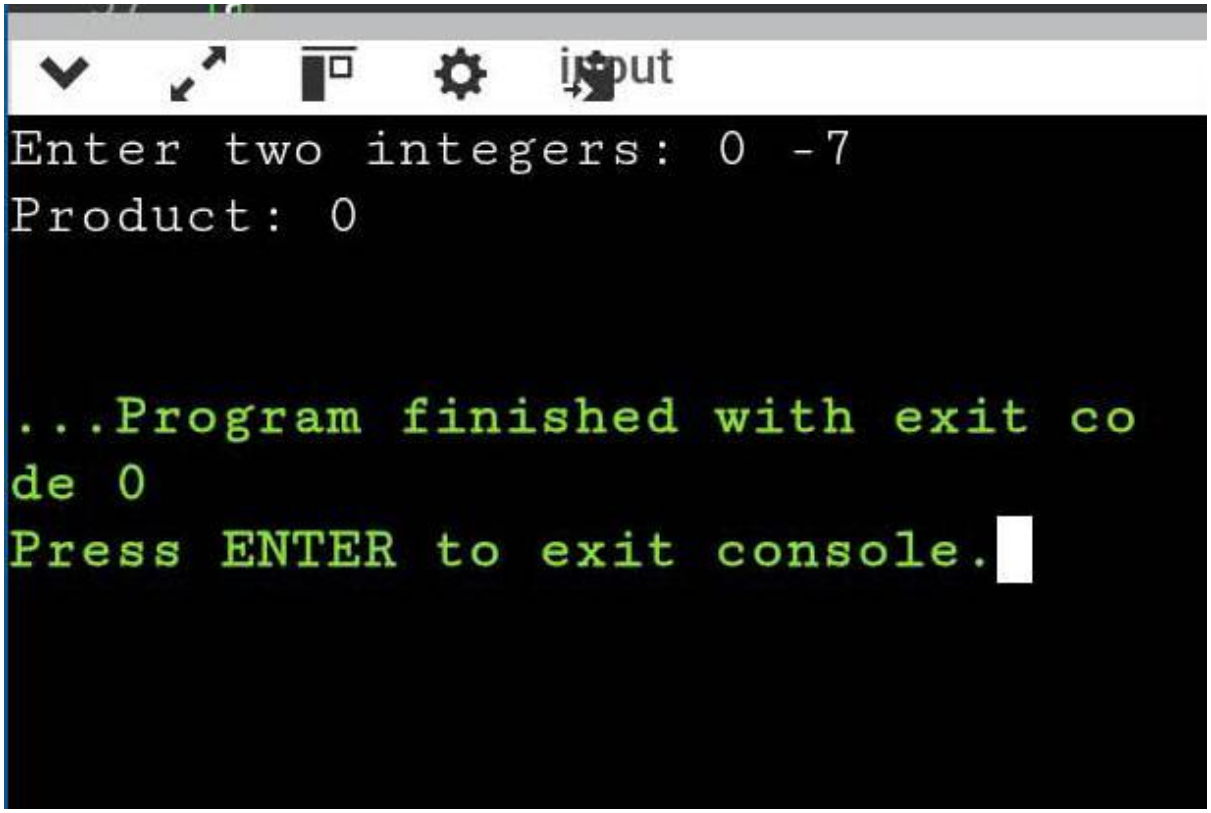
First number is zero.

The other number does not matter.

Product is zero.

Output: 0

Additions: 0



Case 10: -7 0

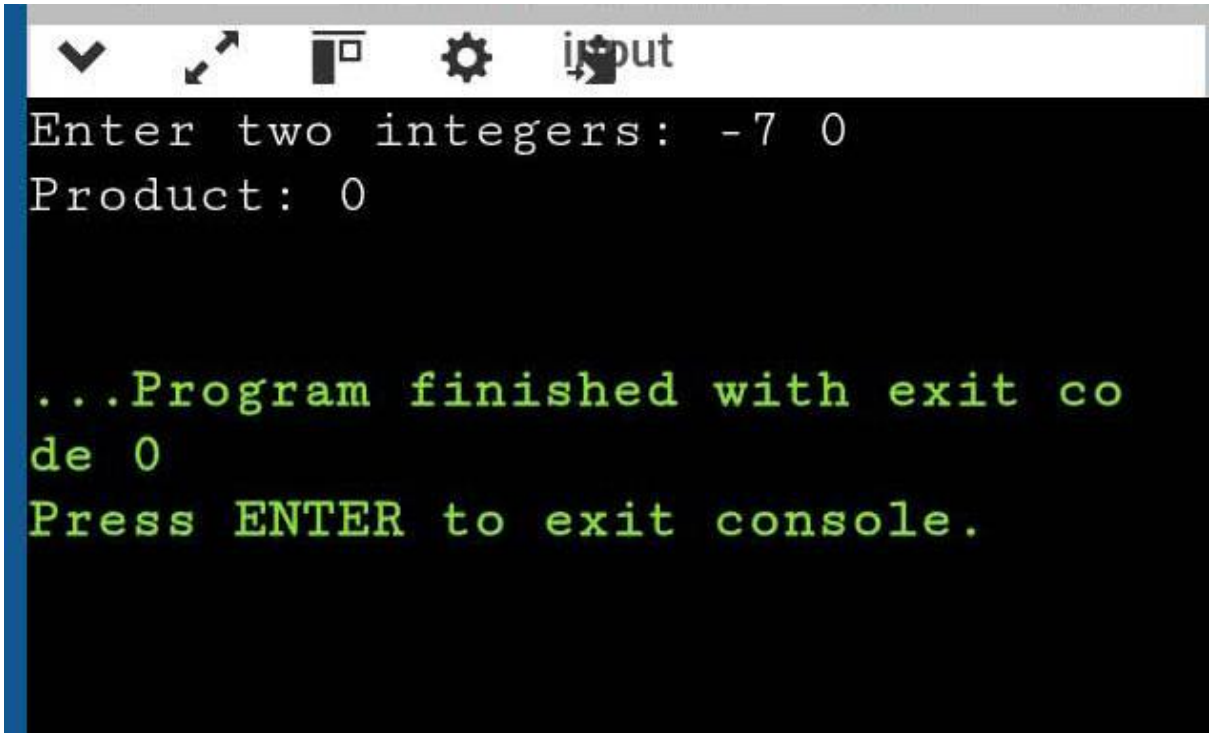
Second number is zero.

Therefore, the product is zero.

No addition is required.

Output: 0

Additions: 0



Case 11: 0 0

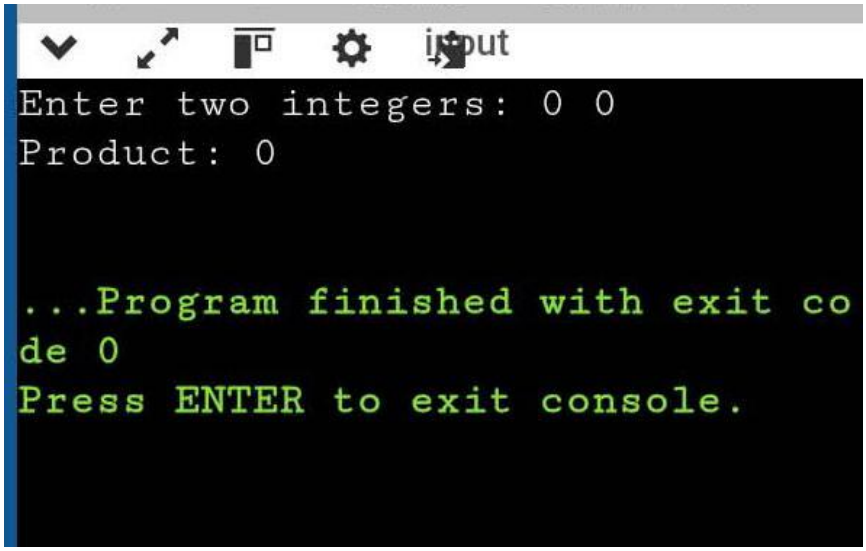
Both numbers are zero.

Product is zero.

No addition is required.

Output: 0

Additions: 0



Case 12: 1 -7

Second number is negative.

We use 7 as the positive value and remember the negative sign.

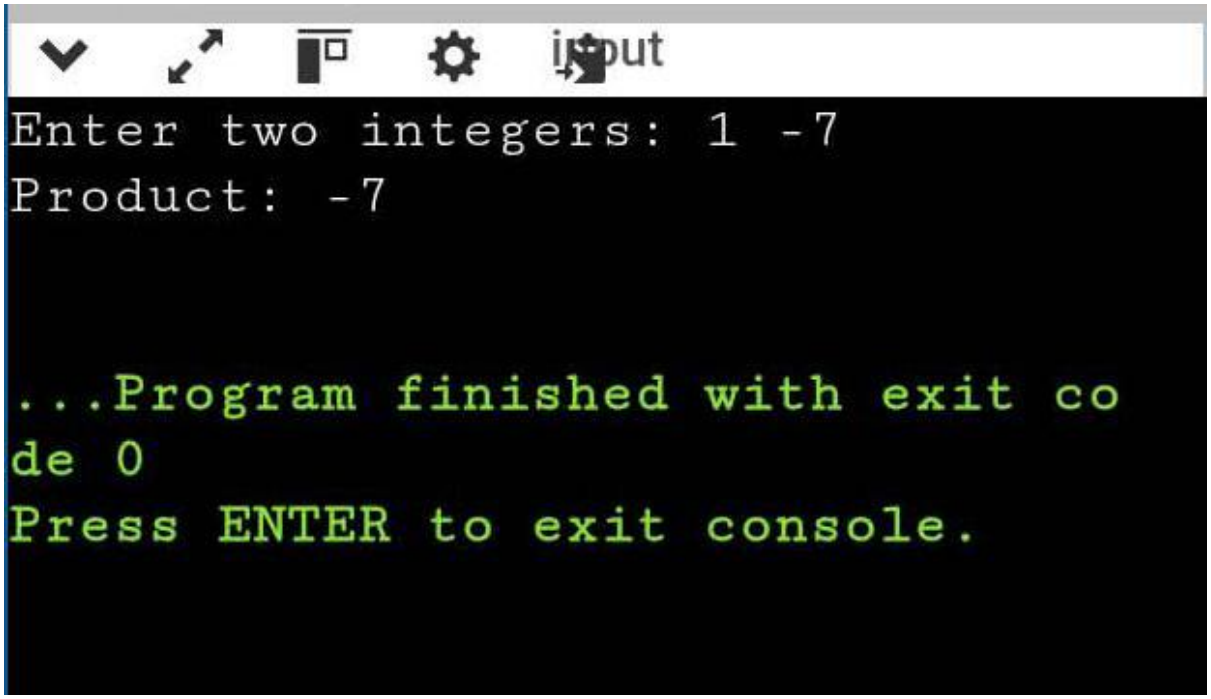
Smaller number is 1, so only one addition is needed.

7 = 7

Final answer is -7.

Output: -7

Additions: 1



Case 13: -1 -9

Both numbers are negative, so the answer is positive.

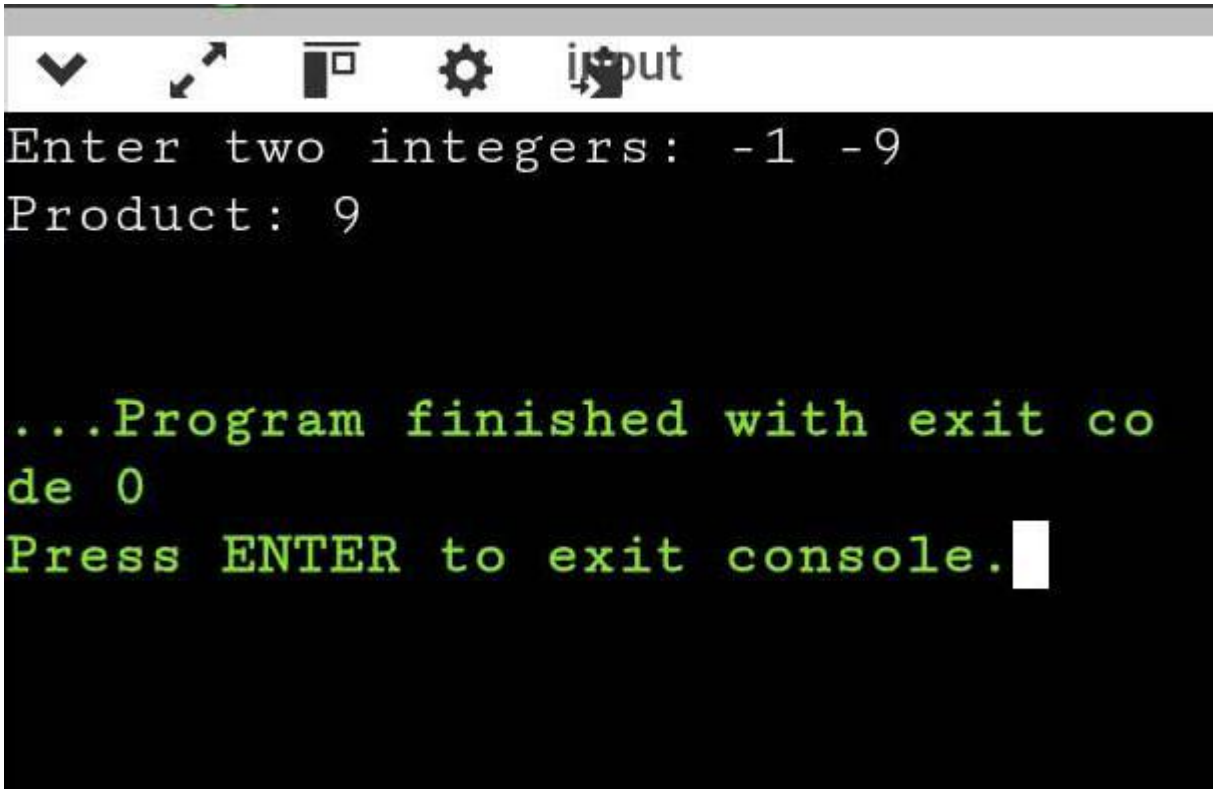
Smaller number is 1.

We add 9 only once.

9 = 9

Output: 9

Additions: 1



Conclusion

The program gives the correct result for all the given test cases. It also handles positive numbers, negative numbers and zero correctly. The program does not use the * operator and uses the smaller number for repeated addition, so the number of additions is minimum.